UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO
Facultad de Ciencias de la Computación
Carrera de Software

Aplicaciones Distribuidas (ISR-701)

TA-IND-01
Informe académico individual

Kubernetes frente a Docker Swarm

Análisis crítico de un sistema distribuido real

Estudiante: Benites Pérez Dariem Alberto
Docente: Gleiston Guerrero Ulloa, PhD
Nivel y paralelo: Séptimo nivel, paralelo B
Fecha: 14 de noviembre de 2026

Repositorio:
`https://github.com/DALBEZ-ROG/AD-TA-IND-01-Benites`

Resumen

Este informe analiza Kubernetes y lo compara con Docker Swarm, que resuelve el mismo problema con un diseño más simple. El análisis aplica los conceptos de la Unidad 1 (transparencias de distribución, CAP y PACELC, modelos de comunicación y tolerancia a fallos) a la documentación oficial, a artículos recientes y a un experimento propio. Levanté un clúster de tres nodos con kind en mi computador, desplegué una aplicación con cuatro réplicas y apagué uno de los nodos trabajadores. Kubernetes marcó el nodo como NotReady a los 50 segundos y esperó otros 300 segundos antes de recrear los pods en el nodo sano; la recuperación completa tomó 5 minutos y 52 segundos, y durante ese tiempo la aplicación funcionó con la mitad de su capacidad. Los dos tiempos coinciden con los valores por defecto documentados. Concluyo que Kubernetes prioriza la consistencia de su estado (etcd con Raft, PC/EC) y prefiere reaccionar tarde antes que reaccionar de más, y que esa prudencia tiene un costo de disponibilidad que el operador debe ajustar según su aplicación. Swarm ofrece garantías parecidas en su plano de control con menos piezas, lo que lo vuelve preferible en despliegues pequeños.

1. Introducción

Cuando una aplicación se divide en contenedores que corren en varias máquinas, algo tiene que decidir dónde corre cada contenedor, reemplazar los que fallan y repartir el tráfico entre las copias. Kubernetes es una plataforma de código abierto que automatiza el despliegue, el escalado y la gestión de aplicaciones en contenedores [1], y se ha convertido en el estándar de facto para orquestarlos [2]. Su idea central es declarativa: el usuario describe el estado que quiere, por ejemplo cuatro réplicas de un servidor web, y el sistema se encarga de mantenerlo, con mecanismos de autorreparación que detectan y recuperan fallos de forma automática [1, p. 1].

Elegí Kubernetes por tres razones. En las fichas de FORM-FB-01 ya había revisado dos estudios sobre su planificador y sobre las mallas de servicios que se montan encima de él [1], [3], así que tenía una base para profundizar. Se puede ejecutar completo en un computador personal con kind, lo que me permitió provocar un fallo real en lugar de describirlo solo con teoría. Y es una opción realista para desplegar aplicaciones como la de nuestro proyecto de fin de curso, que tiene un frontend en Angular, un backend en Spring Boot y una base de datos PostgreSQL.

Para la comparación uso Docker Swarm, el modo de clúster que viene integrado en Docker Engine. Resuelve el mismo problema con otra distribución de responsabilidades: los nodos manager mantienen el estado del clúster, planifican los servicios y atienden la API, mientras que los workers solo ejecutan contenedores [4]. El informe describe la arquitectura de Kubernetes, la analiza con los conceptos de la Unidad 1, presenta mi experimento y termina con la comparación, una postura crítica y las conclusiones.

2. Arquitectura del sistema

Un clúster de Kubernetes tiene dos partes: el plano de control, que toma las decisiones y guarda el estado, y los nodos trabajadores, que ejecutan las aplicaciones. La documentación oficial describe así cada componente [5]:

- **kube-apiserver**: el servidor que expone la API HTTP de Kubernetes. kubectl y los demás componentes trabajan a través de él.
- **etcd**: almacén clave-valor consistente y de alta disponibilidad donde se guardan todos los datos de la API.
- **kube-scheduler**: busca los pods que todavía no tienen nodo y asigna cada uno a un nodo adecuado.

- **kube-controller-manager**: ejecuta los controladores que hacen cumplir el comportamiento de la API, comparando lo pedido con lo que existe.
- En cada nodo, el **kubelet** se asegura de que los pods y sus contenedores estén corriendo, **kube-proxy** mantiene las reglas de red de los Services y el **runtime de contenedores** (containerd en mi clúster) ejecuta los contenedores.

Para estudiarlo armé un clúster con kind (*Kubernetes in Docker*), donde cada nodo es un contenedor de Docker dentro de mi computador. El clúster «benites» tiene un nodo de plano de control y dos trabajadores, los tres con Kubernetes v1.37.0 y containerd 2.3.4 (Figura 1). La Figura 2 muestra los pods del sistema y el nodo en el que corre cada uno: etcd, el apiserver, el scheduler y el controller-manager están en `benites-control-plane`, mientras que kube-proxy y kindnet, la red de los pods, aparecen una vez por nodo. El kubelet y containerd no salen en esa lista porque no son pods: son procesos que corren directamente en cada nodo.

La comunicación forma una estrella alrededor del kube-apiserver. kubectl le envía las órdenes; el scheduler y los controladores consultan y actualizan el estado del clúster a través de él; y el kubelet de cada nodo le avisa que sigue vivo renovando un objeto Lease cada 10 segundos [6], [7]. El estado completo queda guardado en etcd.

La Figura 3 es el diagrama final, hecho en draw.io a partir de mi primer bosquejo a mano (Anexo A). El bosquejo ya tenía los tres nodos, sus componentes y las flechas hacia el kube-apiserver. En la versión final agregué el computador y Docker Desktop como el entorno donde viven los nodos, la función de cada componente debajo de su nombre, el rótulo de los latidos entre el apiserver y los kubelets, la mención de Raft en etcd y una leyenda de colores por rol.

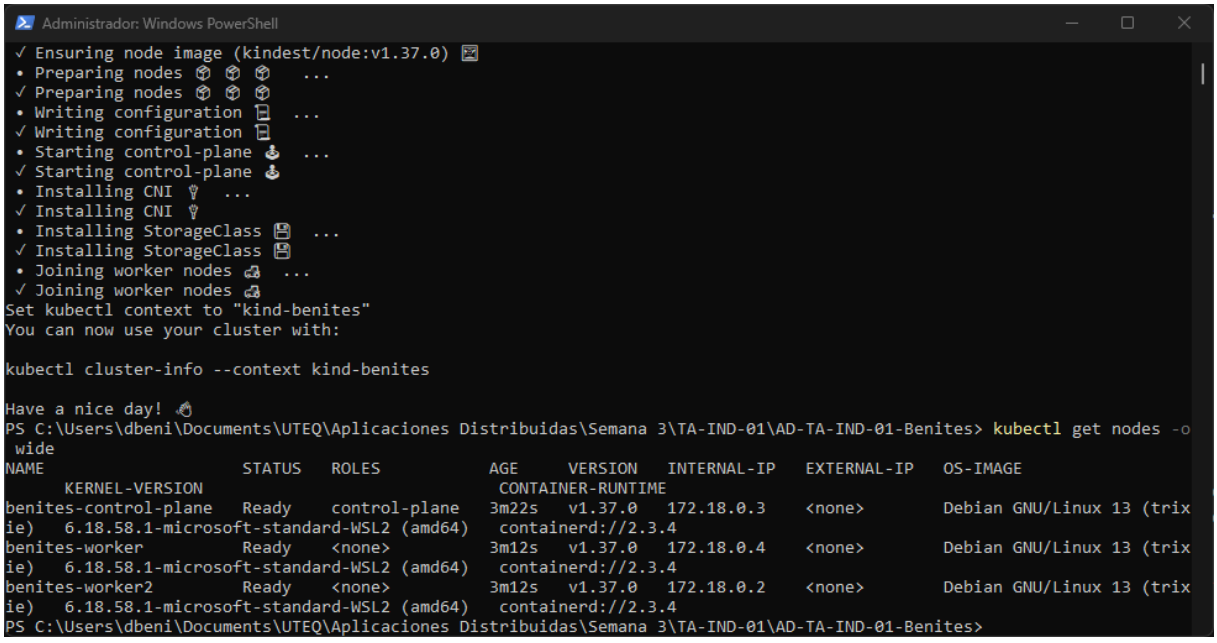


Figura 1: Los tres nodos del clúster «benites» en estado Ready (captura propia, 12/11/2026).

3. Análisis con los conceptos de la Unidad 1

3.1. Transparencias de distribución

Kubernetes ofrece varias transparencias, aunque no todas de forma completa:

- **Acceso**: los clientes usan siempre la misma API HTTP, sin importar en qué nodo esté el recurso que consultan [5].

```
Administrador: Windows PowerShell
ie)   6.18.58.1-microsoft-standard-WSL2 (amd64)   containerd://2.3.4
PS C:\Users\dbeni\Documents\UTEQ\Aplicaciones Distribuidas\Semana 3\TA-IND-01\AD-TA-IND-01-Benites> kubectl get pods -n
kube-system -o wide
NAME                                          READY   STATUS    RESTARTS   AGE     IP           NODE
  NOMINATED NODE   READINESS GATES
coredns-559f6c778d-44dms                      1/1     Running   0          4m28s   10.244.0.3   benites-control-plane
  <none>           <none>
coredns-559f6c778d-vjr98                      1/1     Running   0          4m28s   10.244.0.4   benites-control-plane
  <none>           <none>
etcd-benites-control-plane                    1/1     Running   0          4m37s   172.18.0.3   benites-control-plane
  <none>           <none>
kindnet-h2bph                                 1/1     Running   0          4m28s   172.18.0.2   benites-worker2
  <none>           <none>
kindnet-h8bzj                                 1/1     Running   0          4m28s   172.18.0.4   benites-worker
  <none>           <none>
kindnet-s4s2b                                 1/1     Running   0          4m29s   172.18.0.3   benites-control-plane
  <none>           <none>
kube-apiserver-benites-control-plane          1/1     Running   0          4m37s   172.18.0.3   benites-control-plane
  <none>           <none>
kube-controller-manager-benites-control-plane 1/1     Running   0          4m37s   172.18.0.3   benites-control-plane
  <none>           <none>
kube-proxy-j7rt9                              1/1     Running   0          4m28s   172.18.0.2   benites-worker2
  <none>           <none>
kube-proxy-ncgtz                              1/1     Running   0          4m28s   172.18.0.4   benites-worker
  <none>           <none>
kube-proxy-pwsvm                              1/1     Running   0          4m29s   172.18.0.3   benites-control-plane
  <none>           <none>
kube-scheduler-benites-control-plane          1/1     Running   0          4m37s   172.18.0.3   benites-control-plane
  <none>           <none>
PS C:\Users\dbeni\Documents\UTEQ\Aplicaciones Distribuidas\Semana 3\TA-IND-01\AD-TA-IND-01-Benites>
```

Figura 2: Pods del sistema en `kube-system` y el nodo en el que corre cada uno (captura propia, 12/11/2026).

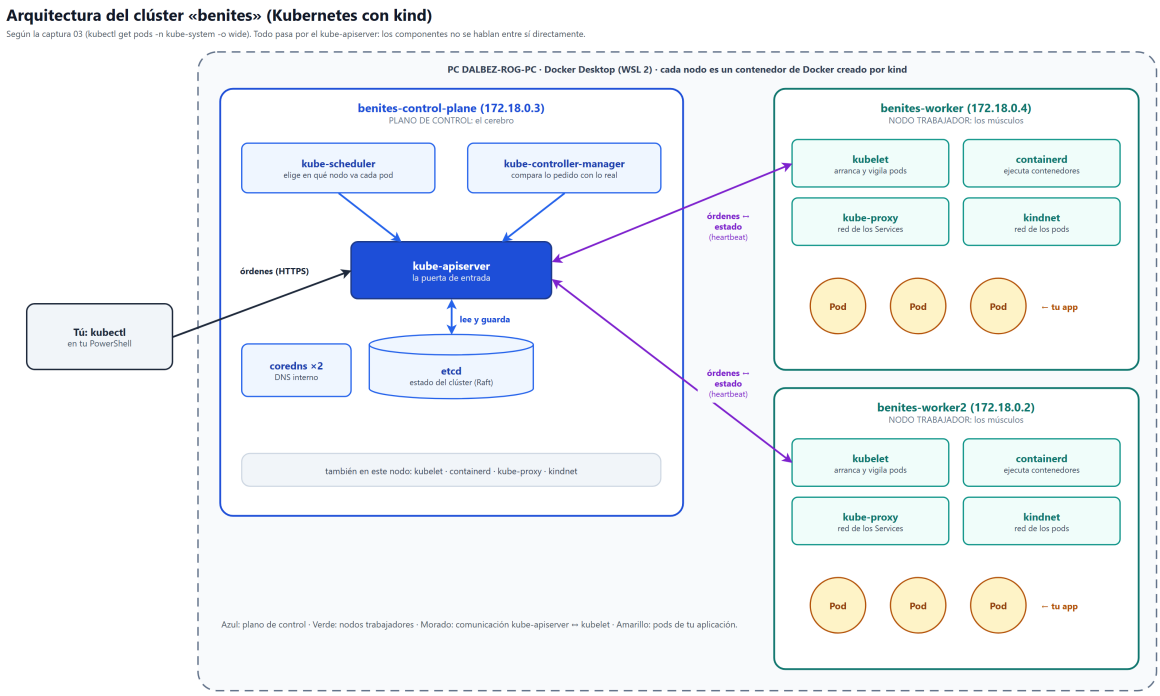


Figura 3: Arquitectura del clúster «benites»: plano de control, nodos trabajadores y comunicación con el kube-apiserver. Elaboración propia en draw.io.

- **Ubicación**: un cliente no necesita saber en qué nodo corre un pod. Kubernetes crea registros DNS para los Services, de modo que se los contacta por un nombre estable y no por una dirección IP [8]. En mi clúster ese DNS lo prestan los dos pods de coredns (Figura 2).
- **Replicación**: el Deployment `web` tiene cuatro réplicas, pero quien lo usa a través de un Service ve un solo servicio.
- **Migración y reubicación**: en mi experimento, los pods del nodo apagado se recrearon solos en el otro trabajador sin que yo indicara dónde (Sección 4). Es una transparencia parcial: los pods nuevos tienen otro nombre y otra dirección IP, y solo un Service oculta ese cambio a los clientes.
- **Concurrencia**: etcd ejecuta las operaciones sobre sus datos de forma estrictamente serializable [9], así que varios clientes pueden modificar el clúster al mismo tiempo y cada cambio se aplica como si ocurriera solo y en orden.
- **Fallos**: es la más débil. En mi experimento la aplicación siguió funcionando, pero durante casi seis minutos con la mitad de las réplicas, y kubectl seguía mostrando como «Running» dos pods que estaban en un nodo apagado.

3.2. CAP y PACELC

El teorema CAP establece que, ante una partición de red, un sistema tiene que elegir entre consistencia y disponibilidad; PACELC agrega que, incluso sin partición, hay un compromiso entre latencia y consistencia [10].

El estado de Kubernetes vive en etcd, que por defecto garantiza linealizabilidad: toda petición linealizada pasa por el proceso de consenso Raft [9]. Raft necesita que una mayoría de los miembros acepte cada cambio [11], de modo que, si una partición deja a una parte del clúster sin mayoría, esa parte deja de aceptar escrituras. Ante una partición, Kubernetes elige consistencia (PC). Sin partición también la prefiere (EC), aunque etcd deja elegir: una lectura en modo serializable se ahorra el costo de pasar por el quórum, a cambio de poder devolver datos desactualizados [9]. Ese es el compromiso entre latencia y consistencia que describe PACELC, y Kubernetes lo resuelve por defecto a favor de la consistencia.

Mi experimento muestra una prudencia parecida en el manejo de los nodos. Cuando `benites-worker` dejó de responder, el plano de control no tenía forma de saber si el nodo se había caído o si solo estaba aislado por la red. En vez de mover los pods de inmediato, esperó 300 segundos. Interpreto esa espera como una decisión de no reaccionar ante un problema pasajero, pagada con disponibilidad: mientras tanto, la aplicación trabajó con la mitad de sus réplicas.

3.3. Modelos de comunicación

Kubernetes sigue un modelo cliente-servidor: el kube-apiserver es el servidor y los demás componentes, junto con kubectl, son sus clientes [5]. Cada petición a la API es sincrónica, porque el cliente espera la respuesta HTTP. Pero el sistema en conjunto trabaja de forma asincrónica: crear un Deployment solo registra el estado deseado, y después el scheduler, los controladores y los kubelets, cada uno por su cuenta, lo convierten en pods corriendo. En mi experimento, `kubectl create deployment` respondió de inmediato con `deployment.apps/web created`, y hizo falta `kubectl rollout status` para esperar a que las cuatro réplicas estuvieran listas.

Los nodos se comunican con el plano de control mediante mensajes periódicos: el kubelet renueva su Lease cada 10 segundos [6] y el controlador de nodos revisa el estado de cada nodo cada 5 segundos [7]. Dentro de etcd, los miembros se coordinan intercambiando mensajes con el protocolo Raft [9].

Para la comunicación entre las aplicaciones, Kubernetes ofrece Services con nombres DNS [8], y kube-proxy mantiene en cada nodo las reglas que dirigen el tráfico a las réplicas [5]. Cuando se necesita más control, se agrega una malla de servicios. Nicolas-Plata et al. explican que «a service mesh (e.g., Istio) is focused on the communication and interactions between services within a microservices architecture» [3, p. 7431], y observaron en sus pruebas que Istio depende de Kubernetes para crear las réplicas de los microservicios [3, p. 7435].

3.4. Tolerancia a fallos

Kubernetes tolera fallos de contenedores, de pods y de nodos trabajadores. Dentro de cada nodo, el kubelet se asegura de que los contenedores sigan corriendo [5]; para la caída de un nodo completo actúa el plano de control. La detección se basa en latidos: si un nodo deja de renovar su Lease durante 50 segundos, el valor por defecto de `--node-monitor-grace-period` [12], el controlador pone su condición Ready en `Unknown` y le agrega la marca `node.kubernetes.io/unreachable` [7], [13]. Los pods toleran esa marca durante 300 segundos por defecto, y recién después son expulsados para que el Deployment cree reemplazos en otros nodos [13]. El plano de control también puede tolerar fallos si etcd se despliega con varios miembros, porque Raft sigue funcionando mientras quede una mayoría.

Flora et al. estudiaron estos mecanismos con inyección de fallos y señalan que, aunque las plataformas de orquestación ofrecen capacidades de *failover*, no se había estudiado qué tan efectivas son ante distintos tipos de fallos [14]. Mi experimento cubre uno solo de esos tipos: la caída completa de un nodo.

Un incidente real muestra el límite de esta tolerancia. El 8 de marzo de 2023, un parche de seguridad de systemd, instalado por las actualizaciones automáticas, reinició systemd-networkd en los nodos con Ubuntu 22.04 de Datadog. Al arrancar, el servicio borró reglas de enrutamiento que necesitaba Cilium, la red de los pods, y dejó sin conexión a más del 60 % de sus instancias [15]. Datadog confiaba en el plano de control de Kubernetes para detectar y reemplazar las instancias fallidas. En AWS se reemplazaron solas, pero miles a la vez, lo que retrasó la recuperación de los almacenes de datos que guardaban información en discos locales; en otras nubes las instancias seguían encendidas y no se reemplazaron sin intervención [15]. La replicación protege contra fallos independientes, pero cuando la misma falla golpea a la mayoría de los nodos a la vez, las réplicas caen juntas.

4. Experimentación propia

Objetivo. Comprobar cómo detecta Kubernetes la caída de un nodo trabajador, cuánto tarda en recuperar la aplicación y qué transparencia ofrece mientras tanto.

Entorno. Mi computador (DALBEZ-ROG-PC) con Docker Desktop 4.99 sobre WSL 2, y el clúster «benites» creado con kind 0.33.0 (Kubernetes v1.37.0): un nodo de plano de control y dos trabajadores. La configuración está en `experimento/kind-config.yaml`.

Procedimiento. Los comandos están en `experimento/comandos.txt`.

1. Desplegué la aplicación `web` (nginx:alpine) con cuatro réplicas. El scheduler las repartió dos y dos entre los trabajadores (Figura 4).
2. En una segunda terminal dejé un monitor que cada cinco segundos registraba el estado de `benites-worker` y de cada pod. Su salida completa está en `experimento/salida-experimento.txt`.
3. Apagué el nodo con `docker stop benites-worker`, que equivale a desenchufar esa máquina.
4. Observé la detección, la marca que recibió el nodo y la reubicación de los pods (Figuras 5 y 6).

5. Encendí de nuevo el nodo con `docker start` y revisé si los pods volvían (Figura 7).

Resultados. El Cuadro 1 reúne los tiempos. Las horas salen del monitor y de los eventos que registró el propio Kubernetes (`kubectl get events`).

Tabla 1: Tiempos medidos en el experimento del 13/11/2026.

Momento	**Hora**	**Qué pasó**
Despliegue	14:55:35	El scheduler asigna dos pods a cada trabajador.
T0	15:04:21	`benites-worker` termina de apagarse (la orden se dio a las 15:04:06).
T1	15:05:11	Evento `NodeNotReady`: el nodo queda marcado como caído.
T2	15:10:11	Evento `TaintManagerEviction`: los dos pods del nodo caído se marcan para borrar y se crean dos nuevos en `benites-worker2`.
T3	15:10:13	El monitor muestra las cuatro réplicas en Running en `benites-worker2`.
Retorno	15:11:51	El nodo vuelve a Ready; los pods no regresan.

Detección (T1 − T0): **50 s**. Espera antes de reubicar (T2 − T1): **300 s**. Recuperación total (T3 − T0): **5 min 52 s**.

Interpretación. Los dos tiempos principales coinciden exactamente con los valores documentados. Los 50 segundos de detección son el valor por defecto de `--node-monitor-grace-period` [12]. Como el kubelet renueva su Lease cada 10 segundos [6], el controlador deja pasar varios latidos perdidos antes de declarar caído el nodo, para no confundir un retraso pequeño de la red con una caída. Los 300 segundos entre el NotReady y la expulsión de los pods son la tolerancia que Kubernetes agrega automáticamente a la marca `unreachable` [13]; la Figura 6 muestra esa marca y la condición Ready en `Unknown` con el mensaje «Kubelet stopped posting node status».

El costo de esa prudencia fue que la aplicación pasó 5 minutos y 52 segundos con dos de sus cuatro réplicas, es decir, con la mitad de su capacidad. Durante ese lapso kubectl seguía mostrando como Running los dos pods del nodo apagado: el plano de control solo conocía el último estado que ese nodo había reportado. Después de la reubicación, los pods viejos quedaron en Terminating y desaparecieron cuando el nodo volvió. Por último, al recuperarse el nodo, los cuatro pods se quedaron en `benites-worker2`. El scheduler solo asigna nodo a los pods que todavía no lo tienen [5], así que no mueve los que ya están corriendo: el clúster queda desbalanceado hasta que alguien intervenga.

5. Ejemplos de mi entorno

SGA de la UTEQ. En época de matrícula el SGA se pone muy lento, y algunas veces, en plena clase, no responde justo cuando un docente quiere descargar un recurso. No ocurre seguido, pero me ha pasado. Como usuario lo veo como un problema de disponibilidad: cuando muchos estudiantes entran a la vez, el sistema no alcanza a atender a todos, y cuando falla no hay nada que oculte la falla. No sé cómo está construido el SGA por dentro, pero en los momentos de más carga no se comporta como un sistema con réplicas suficientes; es lo contrario de mi experimento, donde la aplicación siguió respondiendo con las réplicas que quedaban.

Banca móvil de Banco Pichincha. Varias veces, al hacer una transferencia, la aplicación la mostró «en proceso» durante un rato antes de confirmarla. Lo interpreto como comunicación asincrónica: la aplicación acepta mi orden y me responde enseguida, pero el movimiento del dinero se completa después. En términos de PACELC, me deja seguir usando la aplicación sin esperar, es decir, con baja latencia, y para no mostrarme un dato que todavía no es definitivo, me indica que la operación no está confirmada.

WhatsApp. A veces un mensaje aparece con un aviso de que se está demorando y que llegará pronto, y efectivamente llega más tarde: es mensajería asincrónica, en la que el mensaje

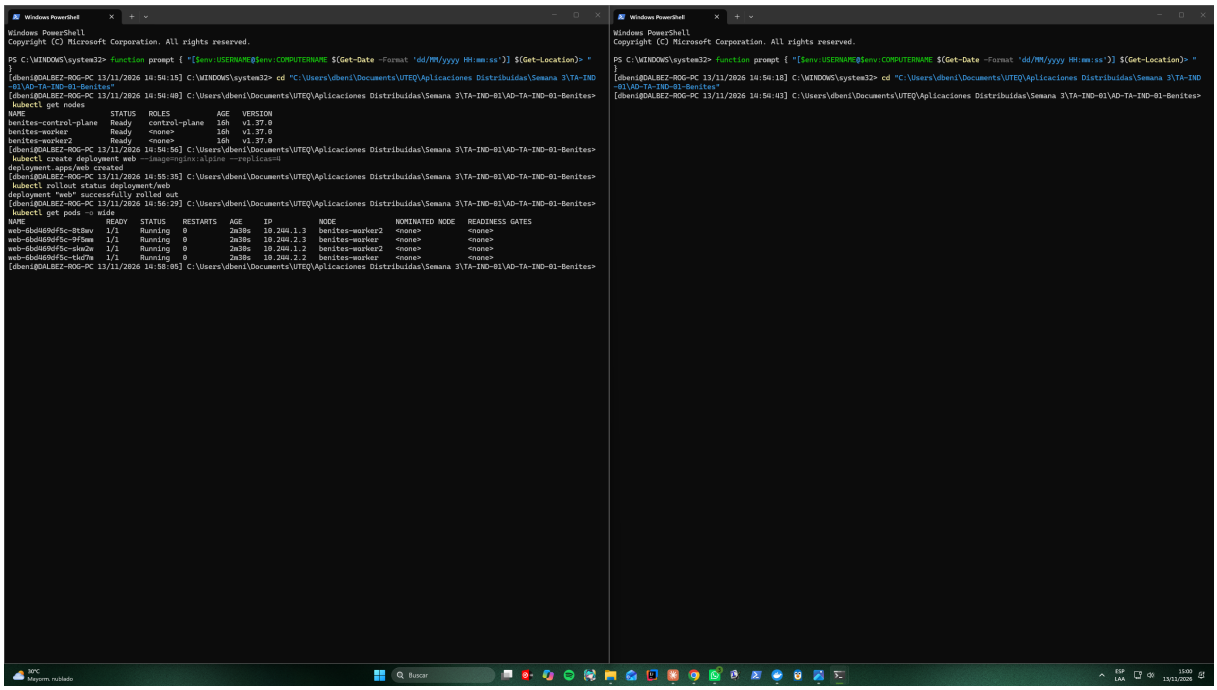


Figura 4: La aplicación `web` con cuatro réplicas repartidas entre los dos trabajadores (captura propia, 13/11/2026).

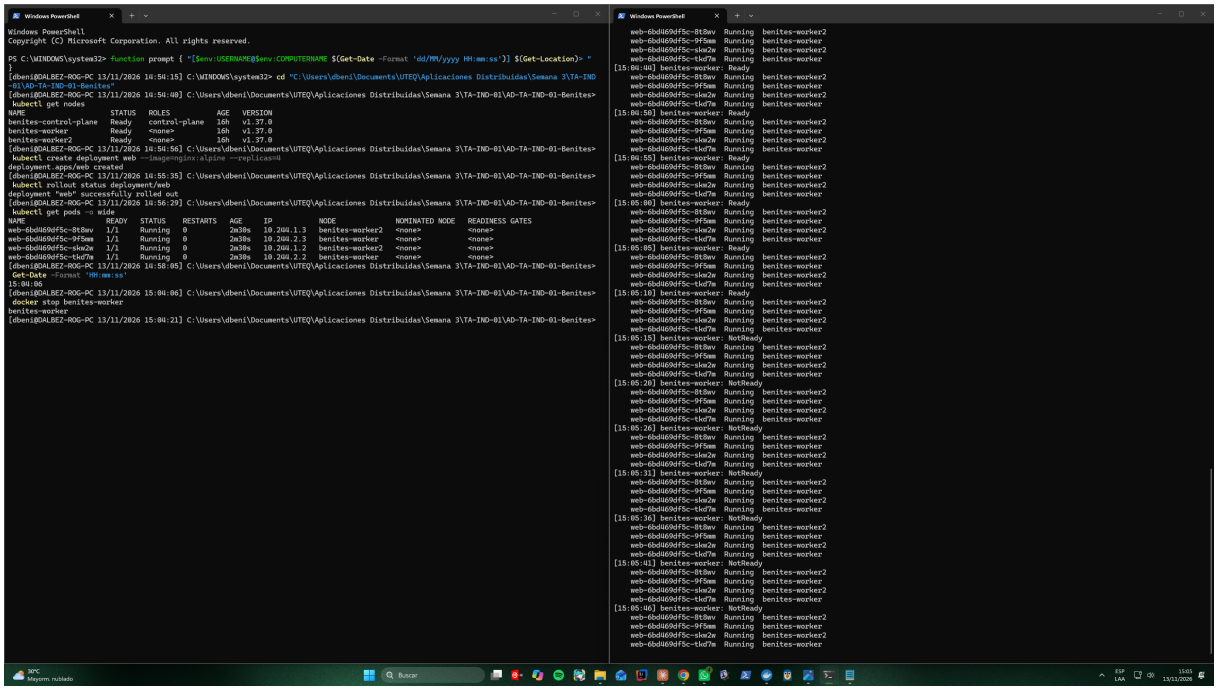


Figura 5: A la izquierda, `docker stop benites-worker`; a la derecha, el monitor cambia a NotReady a las 15:05:15 (captura propia, 13/11/2026).

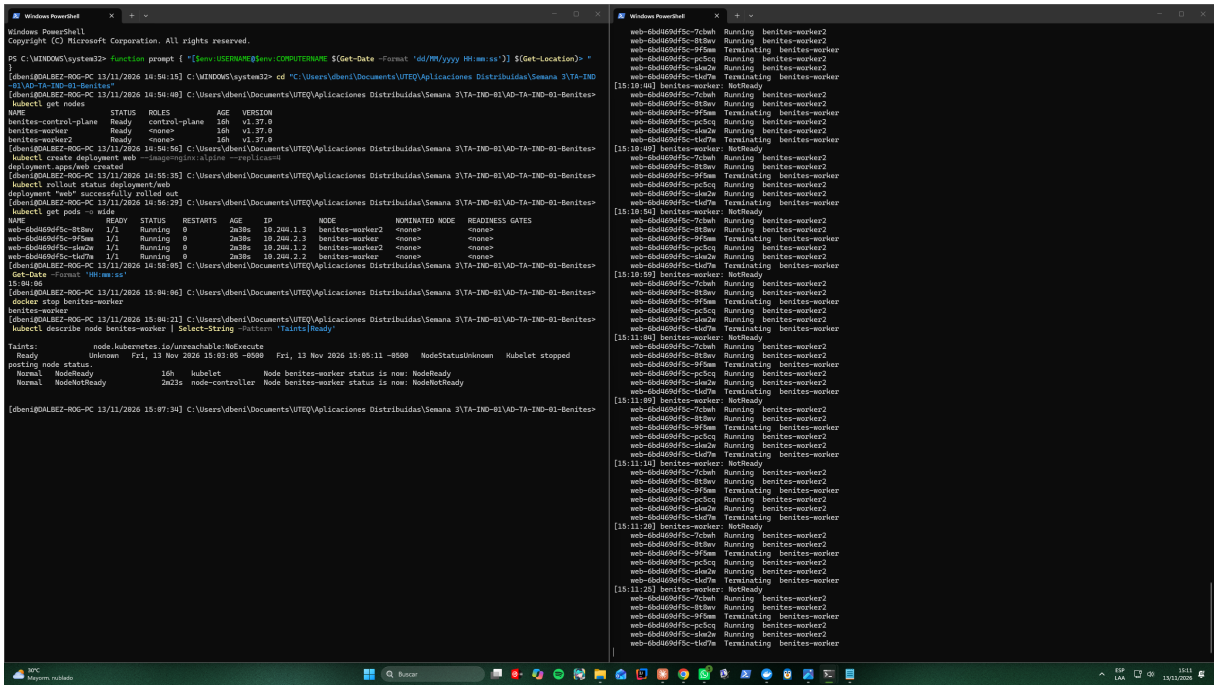


Figura 6: A la izquierda, la marca `unreachable:NoExecute` y la condición Ready en Unknown; a la derecha, dos pods nuevos Running en `benites-worker2` y los dos viejos en Terminating (captura propia, 13/11/2026).

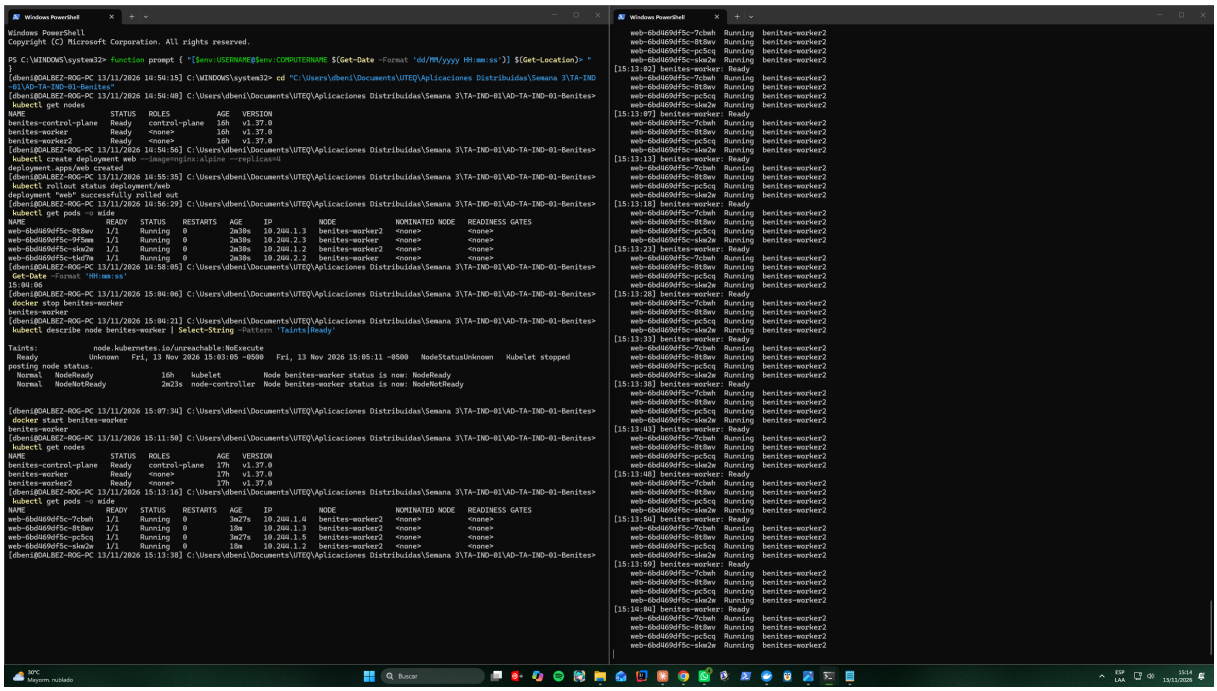


Figura 7: El nodo vuelve a Ready, pero las cuatro réplicas se quedan en `benites-worker2` (captura propia, 13/11/2026).

se entrega cuando puede llegar a su destino. También me ha pasado que WhatsApp falla a medias: los mensajes de texto se envían, pero no las imágenes, los videos ni los stickers. Eso es una falla parcial, en la que una parte del servicio deja de funcionar mientras la otra sigue. Es la misma idea de aislar fallos que tienen los sistemas divididos en servicios, como los que orquesta Kubernetes, donde la caída de un componente no tiene por qué tumbar a los demás.

6. Análisis comparativo

El Cuadro 2 compara Kubernetes con Docker Swarm usando criterios de la Unidad 1.

Tabla 2: Kubernetes frente a Docker Swarm.

Criterio	**Kubernetes**	**Docker Swarm**
Modelo y roles	Plano de control (apiserver, etcd, scheduler, controladores) y nodos con kubelet [5]	Managers que guardan el estado, planifican y sirven la API; workers que solo ejecutan contenedores [4]
Estado y consenso	etcd, linealizable por defecto, con Raft [9]	Raft integrado en los managers [11]
CAP y PACELC del plano de control	PC/EC; las lecturas serializables cambian consistencia por latencia [9]	PC: sin quórum no se planifican tareas nuevas, aunque las que corren siguen [11]
Tolerancia del plano de control	Mientras etcd conserve mayoría	Hasta $(N-1)/2$ managers caídos [11]
Caída de un nodo trabajador	Detección en 50 s y reubicación 300 s después, medido en mi experimento [12], [13]	Reprograma las tareas del nodo no disponible en otros nodos [16]; no lo medí
Transparencia de ubicación	DNS para los Services y reglas de kube-proxy [5], [8]	DNS interno por servicio y entrada por cualquier nodo del swarm [16]
Comunicación de control	API HTTP central en el kube-apiserver [5]	API HTTP servida por los managers [4]

Los dos sistemas coinciden en lo esencial: guardan su estado con Raft y, ante una partición, prefieren consistencia. La diferencia está en cómo reparten el trabajo. Kubernetes separa el almacén de estado (etcd) del resto del plano de control y divide las tareas en varios componentes; solo en mi clúster pequeño, de tres nodos, corrían doce pods del sistema (Figura 2). Swarm integra el estado y la planificación en los propios managers, dentro de Docker Engine, y por eso tiene menos piezas que administrar. A cambio, Kubernetes documenta con precisión sus tiempos de detección y de expulsión y permite ajustarlos; en Swarm la documentación dice que las tareas se reprograman, pero no medí en cuánto tiempo, y esa es una limitación de mi comparación. Ninguno de los dos está libre de fallos: Higgins et al. desarrollaron una herramienta de ingeniería del caos justamente porque las aplicaciones en Docker Swarm sufren fallas de memoria, de aplicación y de red que hay que provocar para encontrarlas [17].

7. Postura crítica

La mayor fortaleza de Kubernetes es su modelo declarativo con autorreparación [1, p. 1]: le pedí cuatro réplicas, perdí una máquina y el sistema volvió a tener cuatro sin que yo hiciera nada. También es predecible: los dos tiempos que medí son exactamente los valores por defecto documentados, y se pueden ajustar. Su estado vive en un almacén linealizable, lo que evita decisiones tomadas con datos contradictorios.

Las limitaciones aparecieron en el mismo experimento. Los valores por defecto son conservadores: casi seis minutos con la mitad de la capacidad pueden ser aceptables para una aplicación interna, pero no para un sistema de pagos. Reducir la tolerancia a la marca `unreachable` acelera la recuperación, pero hace que un corte breve de red provoque movimientos de pods innecesarios; ese es el compromiso que el operador tiene que decidir según su aplicación. Durante la falla, kubectl mostraba como Running pods que ya no existían, lo que puede confundir a quien diagnostica. Y después de la recuperación el clúster quedó desbalanceado, porque nadie mueve de vuelta los pods.

El caso de Datadog muestra un límite más profundo: la replicación supone fallos independientes, y una misma actualización dejó sin red a más del 60 % de los nodos [15]. La reparación automática tampoco es siempre buena: reemplazar miles de instancias a la vez borró datos locales y retrasó la recuperación.

Kubernetes no es la mejor opción para sistemas pequeños. Blinowski et al. encontraron que «on a single machine, a monolith performs better than its microservice-based counterpart» y concluyen que la arquitectura de microservicios no es la más adecuada para todos los contextos [18, p. 20372]. Un sistema como el de nuestro proyecto de fin de curso, para una sola universidad y con pocos usuarios simultáneos, funciona bien en un servidor o con Docker Swarm; Kubernetes se justifica cuando hay muchos servicios, muchos nodos y la necesidad de recuperarse sin intervención humana.

8. Conclusiones

- Kubernetes detectó la caída de un nodo en 50 segundos y esperó 300 segundos más antes de reubicar los pods. Los dos valores coinciden con los documentados, así que su comportamiento ante fallos es predecible y configurable.
- La transparencia de fallos es parcial por diseño: la aplicación siguió funcionando, pero 5 minutos y 52 segundos con la mitad de las réplicas. Para aplicaciones críticas hay que ajustar la tolerancia o distribuir más réplicas en más nodos.
- El plano de control prioriza la consistencia: etcd es linealizable por defecto y usa Raft, por lo que Kubernetes se ubica como PC/EC en PACELC.
- Después de un fallo, Kubernetes no rebalancea los pods; esa tarea queda en manos del operador.
- Swarm ofrece garantías parecidas en su plano de control con menos componentes, lo que lo vuelve preferible en despliegues pequeños; Kubernetes conviene cuando la escala justifica su complejidad.

Declaración de uso de inteligencia artificial

Herramienta: Claude (Anthropic), modelo Claude Opus 5.5, en la aplicación Claude Code.

Para qué se usó: Instalar kind y kubectl y diagnosticar la instalación de WSL y Docker. Preparar el procedimiento del experimento y el script del monitor, que yo ejecuté en mi computador. Buscar las fuentes y verificar sus datos. Hacer una imagen de referencia de la arquitectura, que redibujé a mano y en draw.io, y exportar mi diagrama de draw.io a imagen. Redactar el borrador del informe a partir de mis datos (capturas, tiempos medidos y esquema) y armar el LaTeX. Ayudar con el borrador de la bitácora.

Cómo se verificó: Cada DOI se comprobó en Crossref y cada página de documentación se abrió para confirmar lo citado. Los tiempos del informe se compararon con mis capturas, con `experimento/salida-experimento.txt` y con los eventos de Kubernetes. Leí, corregí y verifiqué todo el texto antes de entregarlo, y lo explico sin leer en el video del README.

Referencias

[1] K. Senjab, S. Abbas, N. Ahmed, and A. u. R. Khan, "A survey of Kubernetes scheduling algorithms," *Journal of Cloud Computing*, vol. 12, no. 1, 2023, Art. no. 87. DOI: 10.1186/s13677-023-00471-1

[2] C. Carrión, "Kubernetes Scheduling: Taxonomy, Ongoing Issues and Challenges," *ACM Computing Surveys*, vol. 55, no. 7, pp. 1–37, Jul. 2023. DOI: 10.1145/3539606

[3] A. Nicolas-Plata, J. L. Gonzalez-Compean, and V. J. Sosa-Sosa, "A service mesh approach to integrate processing patterns into microservices applications," *Cluster Computing*, vol. 27, no. 6, pp. 7417–7438, 2024. DOI: 10.1007/s10586-024-04342-5

[4] Docker Inc. "How nodes work," Docker Docs, Accessed: Nov. 13, 2026. [Online]. Available: https://docs.docker.com/engine/swarm/how-swarm-mode-works/nodes/

[5] The Kubernetes Authors. "Kubernetes Components," Kubernetes Documentation, Accessed: Nov. 13, 2026. [Online]. Available: https://kubernetes.io/docs/concepts/overview/components/

[6] The Kubernetes Authors. "Kubelet Configuration (v1beta1)," Kubernetes Documentation, Accessed: Nov. 13, 2026. [Online]. Available: https://kubernetes.io/docs/reference/config-api/kubelet-config.v1beta1/

[7] The Kubernetes Authors. "Nodes," Kubernetes Documentation, Accessed: Nov. 13, 2026. [Online]. Available: https://kubernetes.io/docs/concepts/architecture/nodes/

[8] The Kubernetes Authors. "DNS for Services and Pods," Kubernetes Documentation, Accessed: Nov. 13, 2026. [Online]. Available: https://kubernetes.io/docs/concepts/services-networking/dns-pod-service/

[9] The etcd Authors. "etcd API guarantees," etcd Documentation v3.6, Accessed: Nov. 13, 2026. [Online]. Available: https://etcd.io/docs/v3.6/learning/api_guarantees/

[10] D. Abadi, "Consistency Tradeoffs in Modern Distributed Database System Design: CAP is Only Part of the Story," *Computer*, vol. 45, no. 2, pp. 37–42, Feb. 2012. DOI: 10.1109/MC.2012.33

[11] Docker Inc. "Raft consensus in swarm mode," Docker Docs, Accessed: Nov. 13, 2026. [Online]. Available: https://docs.docker.com/engine/swarm/raft/

[12] The Kubernetes Authors. “kube-controller-manager,” Kubernetes Documentation, Accessed: Nov. 13, 2026. [Online]. Available: https://kubernetes.io/docs/reference/command-line-tools-reference/kube-controller-manager/

[13] The Kubernetes Authors. “Taints and Tolerations,” Kubernetes Documentation, Accessed: Nov. 13, 2026. [Online]. Available: https://kubernetes.io/docs/concepts/scheduling-eviction/taint-and-toleration/

[14] J. Flora, P. Gonçalves, M. Teixeira, and N. Antunes, “A Study on the Aging and Fault Tolerance of Microservices in Kubernetes,” *IEEE Access*, vol. 10, pp. 132 786–132 799, 2022. DOI: 10.1109/ACCESS.2022.3231191

[15] L. Bernaille. “2023-03-08 incident: A deep dive into the platform-level impact,” Datadog Engineering Blog, Accessed: Nov. 13, 2026. [Online]. Available: https://www.datadoghq.com/blog/engineering/2023-03-08-deep-dive-into-platform-level-impact/

[16] Docker Inc. “Swarm mode key concepts,” Docker Docs, Accessed: Nov. 13, 2026. [También]. Available: https://docs.docker.com/engine/swarm/key-concepts/

[17] T. Higgins, D. N. Jha, and R. Ranjan, “Swarm Storm: An Automated Chaos Tool for Docker Swarm Applications,” in *Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing (HPDC ’24)*, ACM, Jun. 2024, pp. 367–369. DOI: 10.1145/3625549.3658827

[18] G. Blinowski, A. Ojdowska, and A. Przybyłek, “Monolithic vs. Microservice Architecture: A Performance and Scalability Evaluation,” *IEEE Access*, vol. 10, pp. 20 357–20 374, 2022. DOI: 10.1109/ACCESS.2022.3152803

A. Esquema inicial a mano

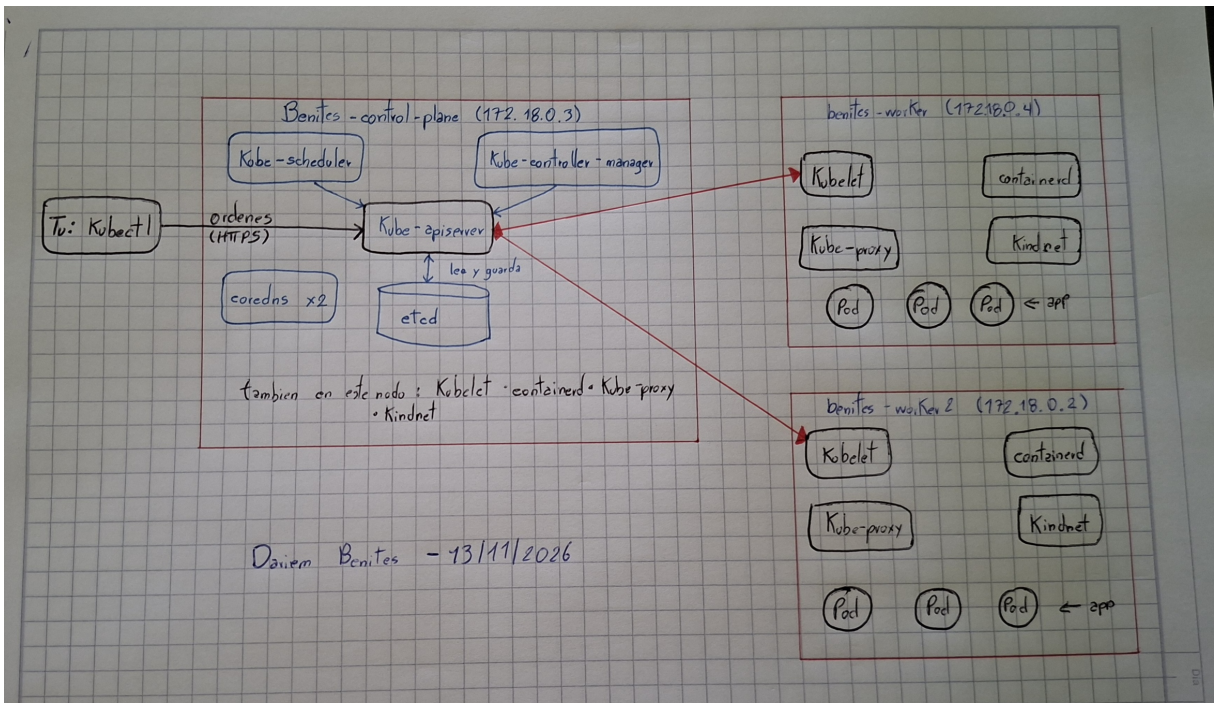


Figura 8: Primer bosquejo a mano de la arquitectura del clúster «benites» (13/11/2026).

B. Capturas de lectura de las fuentes

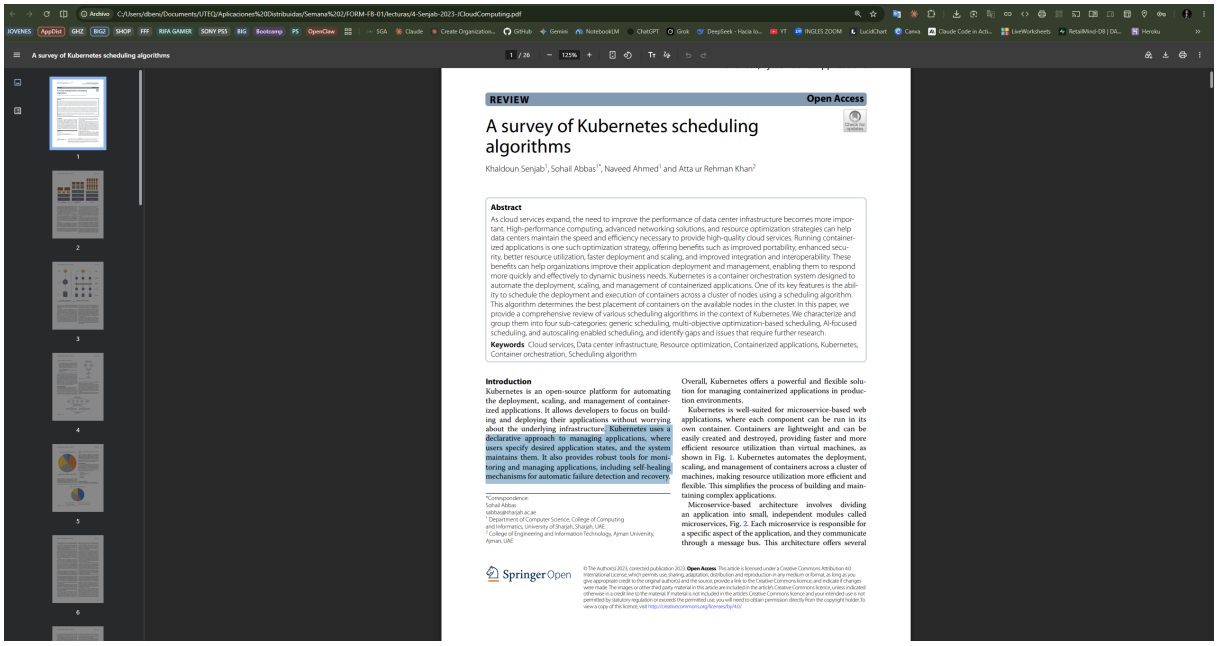


Figura 9: Senjab et al. [1], p. 1: enfoque declarativo y mecanismos de autorreparación.

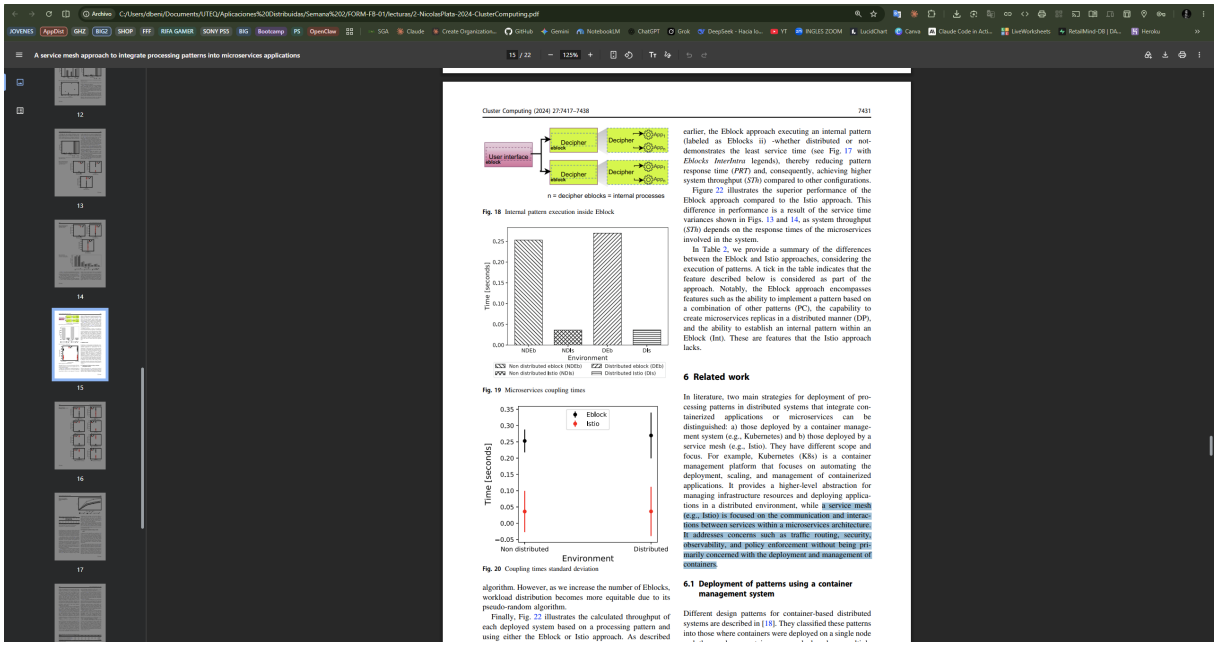


Figura 10: Nicolas-Plata et al. [3], p. 7431: la malla de servicios se ocupa de la comunicación entre servicios.

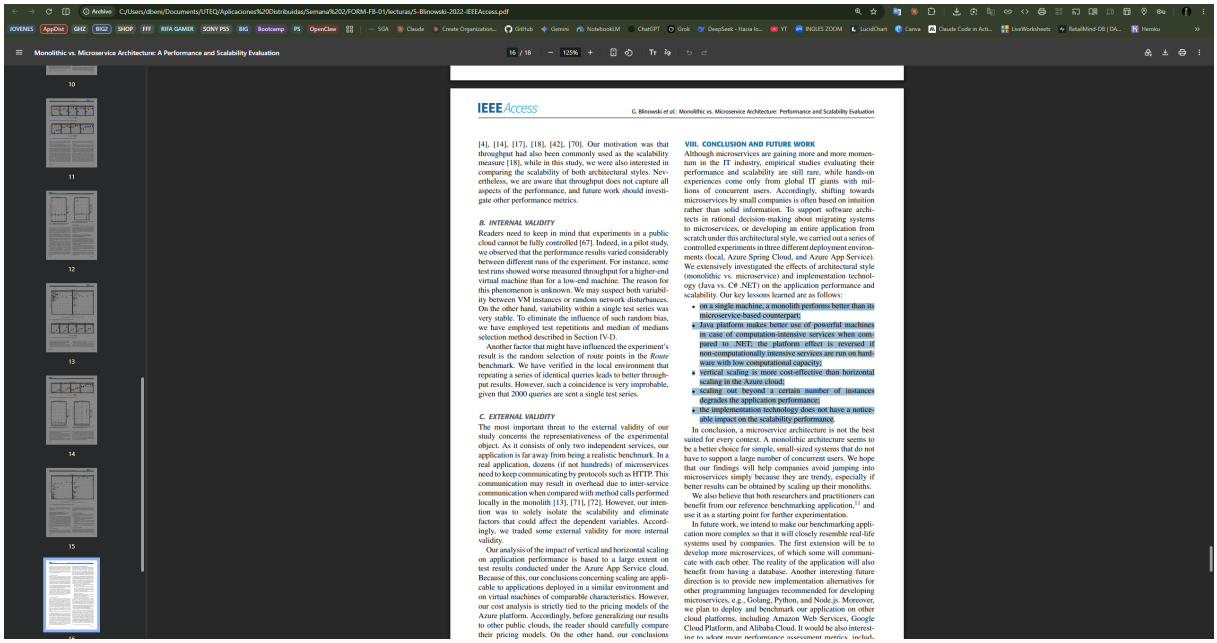


Figura 11: Blinowski et al. [18], p. 20372: en una sola máquina, el monolito rinde mejor.